

DSTI School of Engineering

**Data Pipeline 1 Project: XML-Based Museum and Cultural Heritage
Management Platform**

**Data Pipeline 1
Professor: FARON Catherine**

**Participants:
CASTILLO Veronica
EL FERKH Elias
GIANNINI Benoit
NGUYEN Trang Thuy**

August, 2026

Contents

- 1. Introduction3
- 2. Distribution of the Workload3
- 3. Working Environment and Tools4
- 4. Principles of the models and advantages/disadvantages of the modeling choices4
 - Museum Data-Model Design.....4
 - a. Separate XSD Files Modelling Workaround.....6
- 5. Description of scenarios and solutions.....6
 - Complex Scenario: Exhibitions Displaying Artifacts Made of More Than Five Different Materials6
- 6. Use of AI Tools Report.....7
 - Prompts Used.....7
 - Output Received7
 - Corrections Made.....7
 - How AI Assisted the Project.....7

1. Introduction

The central objective of this project is to construct a robust and coherent Museum XML database capable of supporting non-trivial navigation, filtering and aggregation use cases. XML and XSD are used for data representation and structural validation, while XSLT is mainly used to query and transform the resulting document.

The data model is organized around five principal entities: Themes, Collections, Artifacts, Exhibitions and Restoration Projects. Artifact is the central entity and is connected to the other entities through identifier references. The model combines one-to-many relationships, such as Theme–Artifact, with many-to-many relationships, such as Collection–Artifact and Project–Artifact.

Structural, cardinality and semantic constraints are defined in five XSD files specific to each section, which are then bundled in one Museum.xsd schema. Some semantic rules—such as ensuring that Exhibition Artifacts match the Exhibition Theme—could not be expressed by those schemas, hence they were implemented and verified during dataset generation.

2. Distribution of the Workload

From the beginning of the project, the group agreed on a task-sharing approach that gave every member ownership of the deliverables. The overall contribution of each member, summarized below, was estimated by considering both the number of deliverables each person was individually responsible for and their relative complexity.

Team member	Overall contribution	Tasks contributed to
Veronica	25%	- Took part in the group definition of the XML Schema and of the representative sample XML database. - Designed and implemented three HTML visualization use case scenarios, together with their corresponding XSL stylesheets. - Designed and implemented the YAML export use case scenario and its XSL style sheet. - Contributed to the drafting and revision of the final report.
Trang	25%	- Took part in the group definition of the XML Schema and of the representative sample XML database. - Designed and implemented three HTML visualization use-case scenarios, together with their corresponding XSL stylesheets. - Contributed to the drafting and revision of the final report.
Elias	25%	- Took part in the group definition of the XML Schema and of the representative sample XML database. - Designed and implemented the two use-case scenarios, exporting part of the database into another XML format, together with their XSL style sheets. - Set up and administered the shared GitHub repository used by the group throughout the project. - Contributed to the drafting and revision of the final report.
Benoit	25%	- Took part in the group definition of the XML Schema and of the representative sample XML database. - Designed and implemented the JSON export use-case scenario, its XSL style sheet, and the corresponding JSON Schema. - Implemented the loading, parsing, XML-Schema validation and XSL-stylesheet application program (Python/Java). - Contributed to the drafting and revision of the final report.

Table 1: *Distribution of Workload*

3. Working Environment and Tools

To coordinate the work between the four of us, we relied throughout the project on a shared GitHub repository, which acted as our main collaborative tool. All the deliverables of the project were centralized there, which allowed each member to work independently on their assigned part while keeping a single, up-to-date version of the project and a clear history of who contributed what, through commits. The repository is available at the following address:

github.com/Elias-elf/Museum-Data-Structure

For the actual writing of the code, each member worked individually with the code editor they felt most comfortable with, following the approach and examples that had been presented during the course. Some of us, for instance, worked mainly in Notepad++, while others preferred Visual Studio Code. Once a piece of work was ready, it was pushed to the shared GitHub repository, where the rest of the group could review it, comment on it, and integrate it with their own contributions. Finally, the group organized regular working sessions to align on the overall data model and to review each other's contributions.

4. Principles of the models and advantages/disadvantages of the modeling choices

Museum Data-Model Design

The first challenge was to design a museum data model, with the minimal relational data that allows interesting mapping and navigation across a real-world museum environment system. We first organized the data model around five principal entities, then we incorporated the properties and constraints required by the project specification, as shown in the diagram below:

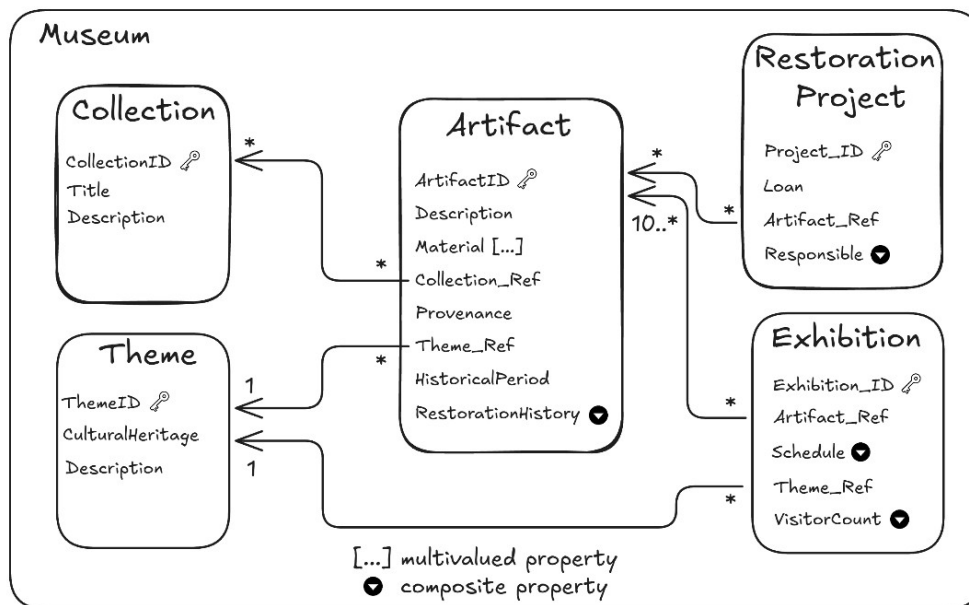


Figure 1: Simplified conceptual data model of the Museum XML document¹

The diagram provides a simplified view of the principal entities and their cardinalities. The exact element structures, datatypes and validation constraints are defined in the corresponding XSD files.

The model avoids unnecessary duplication by assigning unique identifiers to the principal entities and connecting them through references.

Here are the main dependencies and cardinalities (these are commented directly in the corresponding .xsd files):

Many-to-Many
A Collection may contain many Artifacts, and each Artifact belongs to one or more Collections.
A Restoration Project references one or more Artifacts, and an Artifact may be referenced by multiple Projects. The current schema does not model project schedules and therefore cannot determine whether those projects occur simultaneously.

¹ Performed with [Excalidraw](https://excalidraw.com/) (link to the online project)

An Artifact may be displayed in multiple Exhibitions. During generations, the schedules were arranged so that exhibitions displaying the same Artifact do not overlap; this constraint is not enforced by the XSD.
One-to-Many
One Theme covers many Artifacts, but each Artifact has one and only Theme (describing its cultural heritage).
One Theme may be shared by many Exhibitions, but each Exhibition has only one Theme

Table 1: *Main dependencies and cardinalities*

Here are the most prominent semantic rules encoded in the .xsd files:

Artifact
It can be composed of many materials.
The validation file .xsd allows only a limited enumeration of 9 materials, to ensure consistent property content for the generated artifacts and make the filtering easier at the transformation stage (use cases).
For the same reason as above, Artifacts.xsd restricts the Provenance/@type attribute to four categories: Site, City, Country and Region. The geographic value itself remains free text.
Restoration Project
Each Restoration Project must have a Loan of at least €5,000, with the currency fixed to EUR.
The domain of expertise of the project's responsible is limited to 4 categories. Each category will later constitute a common property for all the artifacts covered by the same project.
Exhibitions
Exhibition should contain at least 10 Artifacts

Table 2: *Semantic Rules in XSD*

Here are the semantic rules enforced during generation by AI agent:

Artifact
Artifacts are not arbitrarily built, but are derived from physically existing artifacts to ensure consistency among their properties: for instance, the cultural heritage, the provenance and the historical period should be coherent.
If the artifact is part of a restoration project, then restoration history attribute is "true", an arbitrary record number of past restorations is set and a scope description of the restoration project is provided.
Else, the artifact is not part of any restoration project, thus the scope description property is ignored and an arbitrary record number of past restorations is set (could be zero).
Collection
Theme↔Collection relationship should be loose (many-to-many). In fact, no direct Theme–Collection reference is stored in .xml, but during generation, each Collection should be populated with Artifacts from multiple Themes.
Restoration Project
The Artifacts referenced by a Restoration Project must share the property corresponding to the Responsible specialist's Domain. For example if "Domain" = Material, then all the restored artifacts need to have the same material as a property.
The "Responsible" "Expertise" property needs to be derived from his domain of expertise. For example, if "Domain" = 'Material' then "Expertise" should be 'Marble Expert'.
Exhibitions
Every Artifact referenced by an Exhibition must have the same Theme as that Exhibition.

Table 3: *Semantic Rules for AI Agent*

a. Separate XSD Files Modelling Workaround

One modelling issue that we faced during our contribution to separate XSD files (one pack for each student), was how to preserve five modular XSD sections while validating ID references across sections in a single XML document. XML files cannot contain references to other files and get validated independently. For that purpose, a workaround that we found was to bundle all XSD files in a unique Museum.xsd and validate the global Museum.xml against this schema.

5. Description of scenarios and solutions

Complex Scenario: Exhibitions Displaying Artifacts Made of More Than Five Different Materials

- Objective :

For this scenario, we identify exhibitions displaying artifacts made of more than five distinct materials. The result can help museum managers evaluate material diversity and support conservation planning, since different materials may require specific environmental conditions and preservation methods.

- Data model and main challenge :

The XML data model separates exhibition information from artifact information:

- /Museum/Exhibitions/Exhibition stores exhibition records
- Each exhibition contains artifact references under Artifacts/Artifact/@ref
- The complete artifact records are stored separately under /Museum/Artifacts
- Each artifact contains one or more material elements

Therefore, we had to connect the artifact references in each exhibition with the corresponding artifact records. After that, we collected the materials, remove duplicates and count the remaining distinct values.

- XSLT processing :

To access the complete artifact data throughout the transformation, we stored it in a global variable:

```
<xsl:variable name="artifact-data" select="/Museum/Artifacts"/>
```

Then, we processed all exhibitions: `<xsl:apply-templates select="/Museum/Exhibitions/Exhibition"/>`

The following expression links these references to the artifact records and removes duplicate materials:

```
<xsl:variable name="distinct-materials"
select="$artifact-data/Artifact[@id = $artifact-refs]/Material
[not(. = preceding::Material[parent::Artifact/@id = $artifact-refs])]">
```

Only exhibitions with more than five distinct materials are displayed: `<xsl:if test="count($distinct-materials) > 5">`

An xsl:for-each then sorts and outputs the materials alphabetically in HTML.

- Results :

The transformation identified five qualifying exhibitions (in html file, we have a list of different materials for each exhibition):

Exhibition	Distinct materials
EX008	6
EX018	7
EX025	6
EX029	6
EX041	8

- Evaluation :

The solution calculates results dynamically, without hard-coded exhibition identifiers. It demonstrates references between XML entities, aggregation, duplicate removal, filtering and HTML generation.

Its main limitation is XSLT 1.0 cannot directly process a temporary result tree as a node set. Duplicate removal is case-sensitive: Gold and gold are treated as different values. In this project, consistent material naming is maintained through the XML data and schema constraints.

6. Use of AI Tools Report

As part of the Museum DSTI project, Artificial Intelligence was leveraged to procedurally generate a vast, highly relational XML database. This dataset represents the museum's collections, exhibitions, and restoration projects, serving as the foundational database to test our XSD schemas and XSLT transformations.

Prompts Used

To ensure the generated database was both voluminous and semantically consistent, a strict set of business rules was passed to the AI Agent (ChatGPT Plus, Model 5.6 Sol). The following prompt was used:

"Using those 5 .xsd files, can you generate 1000 artifacts, 30 collections, 12 Themes (means cultural heritage), 50 exhibitions and 100 restoration projects, in separate xml files and under a global Museum.xml file? Please ensure the rules described in the attached table (reference to Table 3 above, Semantic Rules for AI Agent)."

Output Received

The AI successfully generated the interconnected XML files strictly adhering to the requested quantities. The following entities were generated:

- **Artifacts:** 1,000 (originating from retained Art Institute of Chicago catalog records)
- **Collections:** 30 (featuring diverse Theme counts per collection, e.g., COL001 with 9 themes, COL030 with 12 themes)
- **Themes:** 12, **Exhibitions:** 50, **Restoration projects:** 100 (involving 350 specific artifacts)

The outputs passed full business rule validation, confirming that all relationships (such as common domain values for restoration experts, and Theme matching for exhibitions) were accurate.

Corrections Made

Following the generation, validation tests were run to ensure schema compliance. Cross-file ID and IDREF values targeting live references across separate section files (Themes.xml, Collections.xml, etc.) were consolidated and successfully validated together inside the overarching Museum.xml document.

How AI Assisted the Project

The integration of AI tools dramatically accelerated the foundational phase of the project. Generating a realistic, mathematically sound database of over 1,000 coherent entries manually would have been unfeasible within the project's timeframe. This massive dataset was instrumental in safely testing our Python processing pipelines, DOM manipulation scripts, and XSLT style sheets. It allowed the team to verify that the code scales well and handles complex queries against a highly representative museum architecture.